**LAPORAN PRAKTIKUM**

**STRUKTUR DATA**

**MODUL 5**

# Singly Linked List

# (Bagian Kedua)




**Disusun oleh:**

**Rafi Imam Nasrullah**

**109082530010**

**S1IF-13-02**

**PROGRAM STUDI S1 INFORMATIKA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**TAHUN 2026**

# LATIHAN SOAL MODUL

**Link Repo Github Modul X :** https://github.com/RafiImamNasrullah/LAPRAK_RAFI_STRUKTUR-DATA/tree/d3060974e87a79668c6b5e0c3572962c28d3972d/MODUL%205

1. **Guide 1**

   **Source Code**

   **Screenshoot program**

   **Deskripsi program**

2. **Soal 1**

   **Source Code**

**main.cpp**

```
#include <iostream>
#include "SingleLinkedList.h"
using namespace std;

int main() {
   List L;
   ElmList *P1, *P2, *P3, *P4, *P5;

   CreateList(L);

   P1 = alokasi(2);
   insertFirst(L, P1);

   P2 = alokasi(0);
   insertFirst(L, P2);

   P3 = alokasi(8);
   insertFirst(L, P3);

   P4 = alokasi(12);
   insertFirst(L, P4);

   P5 = alokasi(9);
   insertFirst(L, P5);

   cout << "Isi list: ";
   printInfo(L);

   ElmList* P = findElm(L, 8);

   if (P != nullptr) {
```

```
    cout << P->info << " ditemukan dalam list" << endl;
  } else {
    cout << "8 tidak ditemukan dalam list" << endl;
  }

  cout << "Total info dari kelima elemen adalah "
     << sumInfo(L) << endl;

  P = L.First;

  while (P != nullptr) {
    ElmList* Q = P;
    P = P->next;
    dealokasi(Q);
  }

  L.First = nullptr;

  return 0;
}
```

**SingleLinkedList.cpp**

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;

void CreateList(List &L) {
  L.First = nullptr;
}

ElmList* alokasi(int nilai) {
  ElmList* P = new ElmList;
  P->info = nilai;
  P->next = nullptr;
  return P;
}

void dealokasi(ElmList* &P) {
  delete P;
  P = nullptr;
}

void printInfo(List L) {
  ElmList* P = L.First;
```

```
    if (P == nullptr) {
        cout << "List Kosong" << endl;
        return;
    }

    while (P != nullptr) {
        cout << P->info << " ";
        P = P->next;
    }

    cout << endl;
}

ElmList* findElm(List L, int nilai) {
    ElmList* P = L.First;

    while (P != nullptr) {
        if (P->info == nilai) {
            return P;
        }

        P = P->next;
    }

    return nullptr;
}

void insertFirst(List &L, ElmList* P) {
    if (P != nullptr) {
        P->next = L.First;
        L.First = P;
    }
}

int sumInfo(List L) {
    int total = 0;
    ElmList* P = L.First;

    while (P != nullptr) {
        total += P->info;
        P = P->next;
    }
```

```
    return total;
}
```

**SingleLinkedlist.h**

```
#ifndef SINGLYLINKEDLIST_H
#define SINGLYLINKEDLIST_H

struct ElmList {
    int info;
    ElmList* next;
};

struct List {
    ElmList* First;
};

void CreateList(List &L);
ElmList* alokasi(int nilai);
void dealokasi(ElmList* &P);
void printInfo(List L);

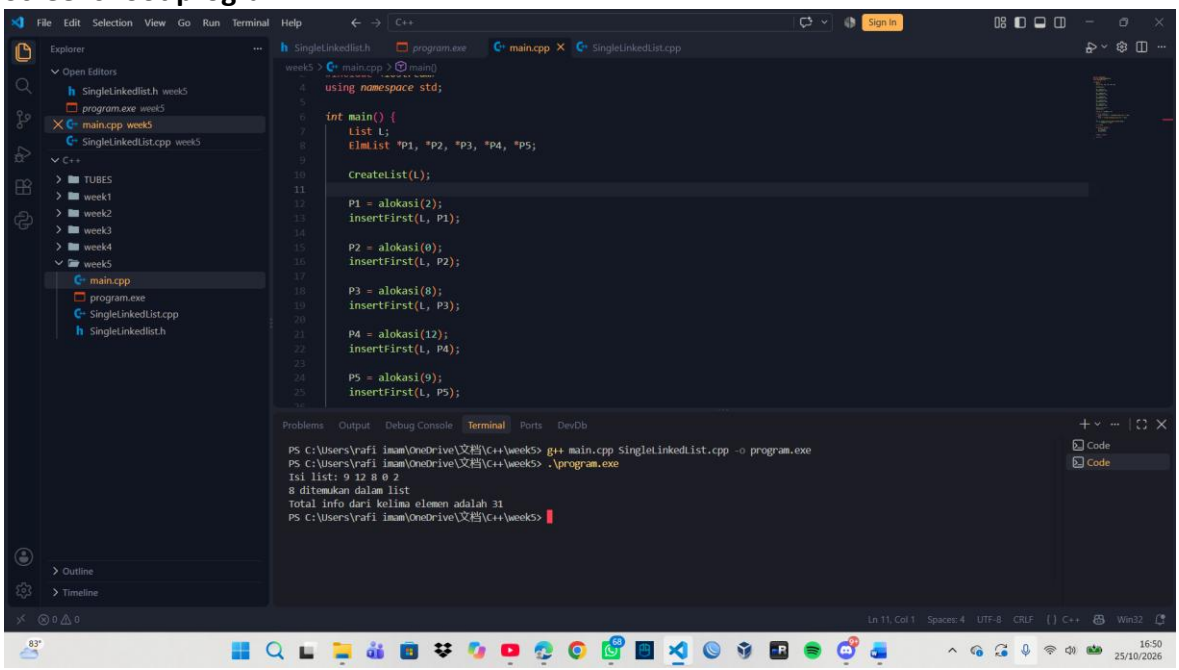
ElmList* findElm(List L, int nilai);

void insertFirst(List &L, ElmList* P);

int sumInfo(List L);

#endif
```

**Screenshoot program**




**Deskripsi program**

Program ini menggunakan fungsi CreateList untuk membuat list kosong, alokasi untuk membuat elemen baru, dan insertFirst untuk menambahkan elemen di awal list. Setelah semua elemen dimasukkan, program menampilkan isi list dengan urutan 9, 12, 8, 0, dan 2. Fungsi findElm digunakan untuk mencari nilai 8 di dalam list dengan memeriksa setiap elemen secara berurutan. Selanjutnya, fungsi sumInfo digunakan untuk menjumlahkan seluruh nilai elemen, yaitu 9, 12, 8, 0, dan 2, sehingga diperoleh total 31.

3. **Soal 2**

   **Source Code**

   **Screenshoot program**

   **Deskripsi program**

4. **Soal 3**

   **Source Code**

   **Screenshoot program**

   **Deskripsi program**

5. **Soal 4**

   **Source Code**

   **Screenshoot program**

   **Deskripsi program**